五、网络部分（李昉老师）

5.1 OSI 七层模型

OSI（开放系统互连）七层模型是计算机网络通信的基础框架，它将网络通信过程划分为七个层次，每个层次都有其特定的功能和协议。以下是 OSI 七层模型各层的协议及其简要说明：

物理层： 负责数据的物理传输，包括电压、电缆规范、中继器、集线器等

主要协议包括 Ethernet、SONET/SDH 等。

数据链路层： 负责将比特流组合成帧，并进行错误检测和纠正

主要协议包括 PPP、HDLC、以太网（Ethernet）、点对点协议（PPP）等。

网络层： 负责路径选择和路由选择

主要协议包括 IP（互联网协议）、ICMP（互联网控制消息协议）、IGMP（互联网组管理协议）等。

传输层： 负责端到端的通信，提供可靠的数据传输服务

主要协议包括 TCP（传输控制协议）、UDP（用户数据报协议）等。

会话层： 负责建立、管理和终止会话

表示层： 负责对数据进行编码和解码，确保数据在不同系统间的正确传输

应用层： 直接为用户提供服务，如文件传输、电子邮件、远程登录等

主要协议包括 HTTP（超文本传输协议）、FTP（文件传输协议）、SMTP（简单邮件传输协议）、DNS（域名系统）等。

5.2 TCP/IP 四层协议族，以及每层有哪些协议，以及各协议的基本功能

TCP/IP 协议族是互联网的核心协议栈，按照功能分为四个层次：链路层、网络层、传输层和应用层。每一层都有特定的协议来处理不同的网络通信任务

链路层：

功能：负责在物理网络上发送数据帧，确保数据在直接连接的网络节点之间可靠传输。链路层协议处理硬件地址（MAC 地址）并管理局域网通信。

主要协议：

华清远见武汉中心嵌入式学科面试宝典

ARP：将 IP 地址解析为 MAC 地址；

RARP：将 MAC 地址解析为 IP 地址；

网络层：

功能：负责在不同网络之间路由数据包，并根据目标 IP 地址选择最佳路径将数据包从源发送到目的地；

主要协议：

IP：是核心协议，负责数据包的路由选择和转发

ICMP：用于发送网络状态和错误信息

IGMP：用于管理多播组成员，用于多播通信

传输层：

功能：提供端到端的通信服务，确保数据在源设备和目标设备之间可靠、顺序传输。传输层处理数据分段、重组、错误检测和流量控制

主要协议：

TCP:提供可靠的、面向连接的传输服务，确保数据包按顺序到达并无丢失。支持三次握手、重传机制和流量控制

UDP:提供无连接的、快速的传输服务，不保证可靠性和顺序性，适用于实时应用如视频流和在线游戏

应用层：

功能：提供用户与网络之间的接口，处理特定的网络应用服务。应用层协议负责数据的格式化、会话管理和应用间的通信。

主要协议：

HTTP:用于网页数据的传输和浏览

FTP:用于文件传输，支持文件上传和下载

SMTP:用于电子邮件的发送和接收

DNS:将域名解析为 IP 地址，支持域名与 IP 地址的映射

SSH:提供安全的远程登录和命令执行

5.3 什么是 IPV4

IPv4（Internet Protocol version 4）是第四版互联网协议，也是目前最广泛使用的互联网协

华清远见武汉中心嵌入式学科面试宝典

议。IPv4 地址是一个 **32 位**的二进制数，通常用**点分十进制**形式表示

地址长度：32 位（4 字节）

地址数量：理论上支持约 42 亿个唯一 IP 地址。

表示形式：点分十进制，例如 192.168.0.1

主要特点：

1. 地址空间有限，随着互联网的快速增长，IPv4 地址逐渐耗尽。
2. 支持广播（Broadcast）通信。

5.4 什么是 IPv6?

IPv6（Internet Protocol version 6）是第六版互联网协议，旨在**替代 IPv4**。IPv6 采用 **128 位**地址，可以提供几乎无限数量的唯一地址，以应对 IPv4 地址耗尽的问题。

地址长度：128 位（16 字节）

地址数量：理论上支持约 3.4×10^{38} 个唯一 IP 地址（相当于每平方米地球表面可以分配上万亿个 IP 地址）。

表示形式：使用冒号分隔的十六进制，例如 2001:0db8:85a3:0000:0000:8a2e:0370:7334

主要特点：

1. 大量地址空间，解决了 IPv4 地址耗尽问题。
2. 内置支持自动配置（Autoconfiguration）。
3. 不再支持传统的广播通信，改用组播（Multicast）和任播（Anycast）。
4. 提供更好的安全性，内置 IPsec 支持。

5.5 IPv4 与 IPv6 的主要区别

1. **地址长度与格式**：

   IPv4：32 位地址，点分十进制表示。

   IPv6：128 位地址，冒号分隔的十六进制表示。

2. **地址空间**：

   IPv4：支持约 42 亿个地址，已接近耗尽。

   IPv6：支持几乎无限数量的地址。

3. **安全性**：

   IPv4：安全性依赖于应用层的协议或附加配置。

华清远见武汉中心嵌入式学科面试宝典

IPv6：内置 IPsec，默认提供更好的安全性。

4. **配置和管理：**

IPv4：依赖手动配置或 DHCP 进行 IP 地址分配。

IPv6：支持自动配置，无需手动配置。

5. **通信模式：**

IPv4：支持广播、组播和单播。

IPv6：不支持广播，只支持组播、任播和单播。

6. **报头结构：**

IPv4：报头较复杂，包含多种字段。

IPv6：简化报头结构，提升路由效率。

5.6 集线器、交换机、路由器工作原理

集线器(Hub)：

工作：在 OSI 模型的**物理层**

功能：主要功能是将多个网络设备连接在一起，并通过**广播方式**在所有端口之间转发数据

原理：广播通信，当一个设备发送数据时，集线器会将该数据包复制并发送到其所有端口。因此，所有连接到集线器的设备都会收到数据包，即使它们不是目标接收者

缺点：因为所有数据包都被广播到每个端口，这会导致网络拥塞和冲突（Collision），尤其是在大规模网络中。此外，集线器无法识别或处理数据包的 MAC 地址。

交换机(Switch)：

工作：在 OSI 模型的**数据链路层**

功能：根据设备的 **MAC 地址**来决定数据包的转发路径

原理：交换机在收到数据包时，会检查数据包中的目标 MAC 地址，并将数据包转发到对应的端口（即目标设备所连接的端口）。通过维护一个 MAC 地址表，交换机可以有效地减少网络流量，避免了集线器的广播问题；交换机支持全双工通信，允许数据在不同端口间同时双向传输，减少了冲突，提高了网络效率。

路由器(Router)：

工作：在 OSI 模型的**网络层**

华清远见武汉中心嵌入式学科面试宝典

功能：负责在不同网络之间进行数据包的路由选择和转发

工作流程：

1. **接收数据包**：路由器从一个网络接口接收到一个 IP 数据包。
2. **检查 IP 头部**：路由器查看数据包的目的 IP 地址。
3. **查找路由表**：路由器查找路由表，决定数据包的下一跳或出口接口。
4. **转发数据包**：路由器将数据包转发到合适的接口，送往下一跳路由器或目的网络。
5. **更新路由表**：如果使用动态路由协议，路由器会根据最新的网络拓扑更新路由表。

5.7 TCP 中三次握手，四次挥手过程

TCP 报文头



源端口号 ：发送方应用程序对应的端口，大小为 16 bit

目的端口号：接收方应用程序对应的端口，大小为 16 bit

序号 seq：数据包的编号，tcp 协议为每个数据包都设置一个 32bit 序号，用于接收方确认

确认序号 ack：应答数据包的编号，用于确认是否收到指定数据包

URG ： 紧急指针标志位，表示紧急指针是否有效

ACK ： 确认标志位，表示确认好是否有效

PSH ： 提示接收端应该立即读取数据

RST ： 重置标志位，表示重新建立连接

SYN ： 同步标志位，表示请求建立连接

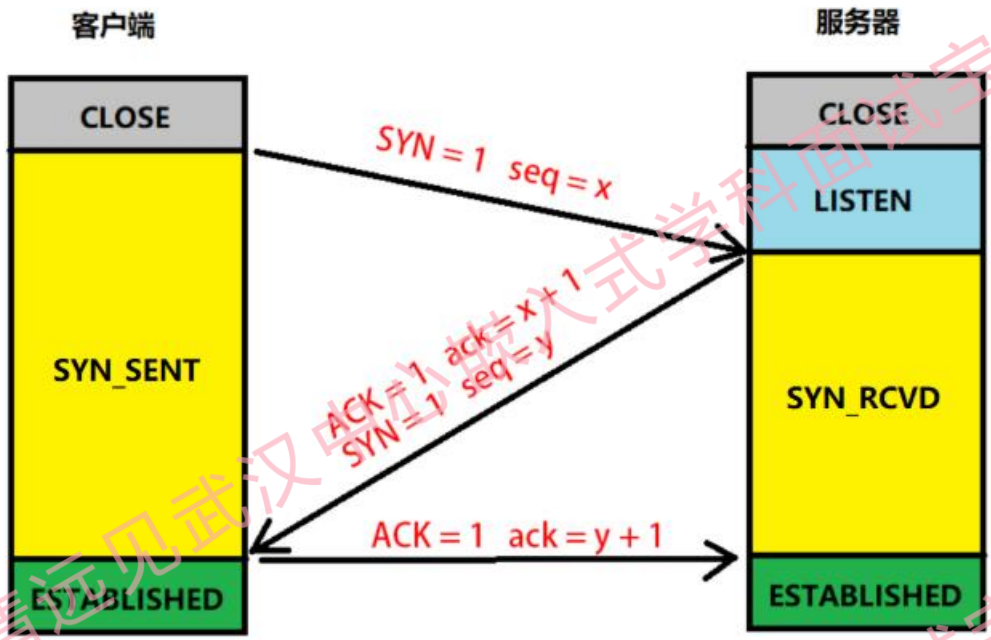
FIN ： 结束标志位，表示请求结束

华清远见武汉中心嵌入式学科面试宝典

三次握手

在 TCP/IP 协议中，TCP 协议提供可靠的连接服务，采用三次握手建立一个连接。

三次握手目的：三次握手确保了连接的双方都同步了彼此的序列号，并准备好传输数据



1. 第一次握手

客户端：向服务器发送一个 SYN 报文，表明客户端希望建立连接，并且告诉服务器客户端的初始序列号 seq

报文内容：包含 SYN 标志位置 1，和客户端选择的初始序列号（如 `seq = x`）

目的：通知服务器客户端要建立连接，并同步序列号

2. 第二次握手

服务器：在收到客户端的 SYN 报文后，服务器回复一个 ACK（回复第一次握手的 SYN，可以理解为 Synchronize-Acknowledgment）；这个报文包括服务器的初始序列号（如 `seq = y`）以及对客户端序列号的确认（ack = x + 1）

报文内容：包含 SYN 标志位和 ACK 标志位都置为 1，`seq = y` 和 `ack = x + 1`

目的：确认收到客户端的连接请求，并同步服务器的序列号

3. 第三次握手

客户端：收到服务器的 SYN-ACK 报文后，客户端向服务器发送一个 ACK 标志位都置为 1 报文，

华清远见武汉中心嵌入式学科面试宝典

确认收到服务器的序列号，并将自己的序列号加 1（`ack = y + 1`）

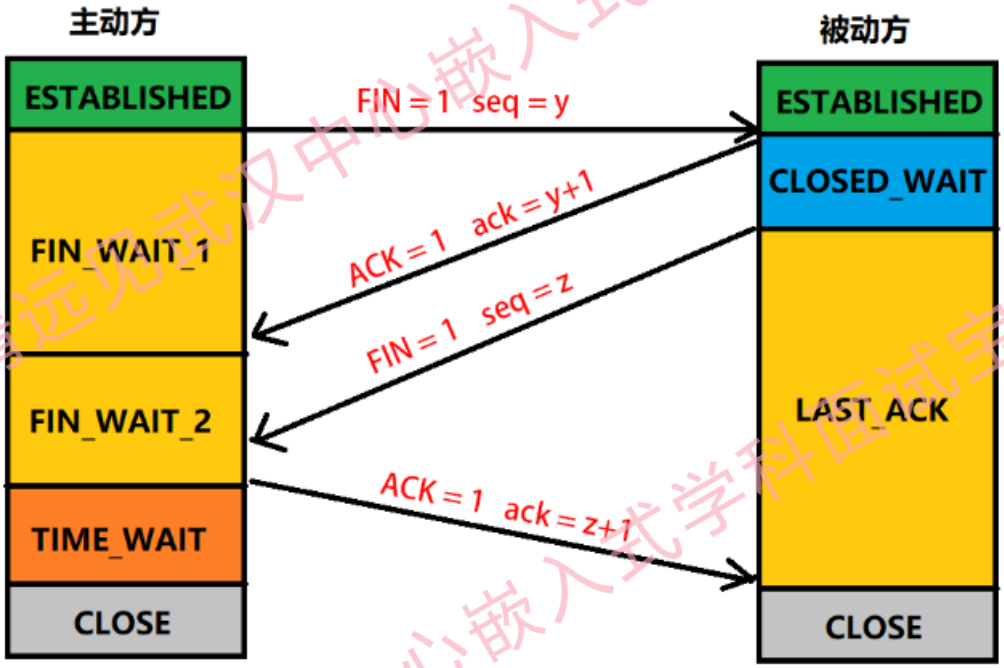
报文内容：包含 ACK 标志位，`Seq = x + 1` 和 `ACK = y + 1`

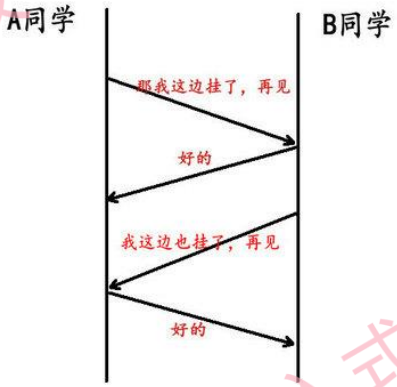
目的：客户端确认已经收到服务器的序列号，完成连接建立的最后一步

四次挥手

由于 TCP 连接是全双工的，因此每个方向都必须单独进行关闭。这好比，我们打电话（全双工），正常的情况下（出于礼貌），通话的双方都要说再见后才能挂电话，保证通信双方都把话说完了才挂电话。

四次挥手目的：可靠地终止 TCP 连接，确保双方都已经完成数据传输，并且确认连接终止。





1. **第一次挥手**

客户端--->服务器：当客户端完成数据发送后，发送一个 FIN 报文，表示想要终止连接

华清远见武汉中心嵌入式学科面试宝典

报文内容：包含 **FIN 标志位置为 1**，和客户端选择的序列号（`seq = y`）

目的：客户端告知服务器没有数据要发送了，准备关闭连接

2. 第二次挥手

服务器--->客户端：服务器收到 FIN 后，回复一个 ACK 报文，确认接收到客户端的 FIN

报文内容：包含 **ACK 标志位置为 1**，和服务器确认序列号（`ack = y + 1`）

目的： 确认收到客户端的终止请求，但服务器可能还有未完成的数据传输

服务器收到挥手消息会做两件事：

第 1 件事：收到客户端的 FIN 时底层会主动回发一个 ACK

第 2 件事：收到客户端的 FIN 时，导致服务器 B 的应用层 read()返回 0（告诉服务器应用层：客户端关闭了）

3. 第三次挥手

服务器--->客户端：服务器在完成数据传输后，发送 FIN 报文，表示也准备关闭连接

报文内容：包含 **FIN 标志位置为 1**，和服务器选择的序列号（`seq = z`）

目的： 服务器告知客户端准备关闭连接

4. 第四次挥手:ACK 置 1

客户端--->服务器:客户端收到服务器的 FIN 后，回复一个 ACK 报文，确认服务器的 FIN（`ACK = Seq + 1`）

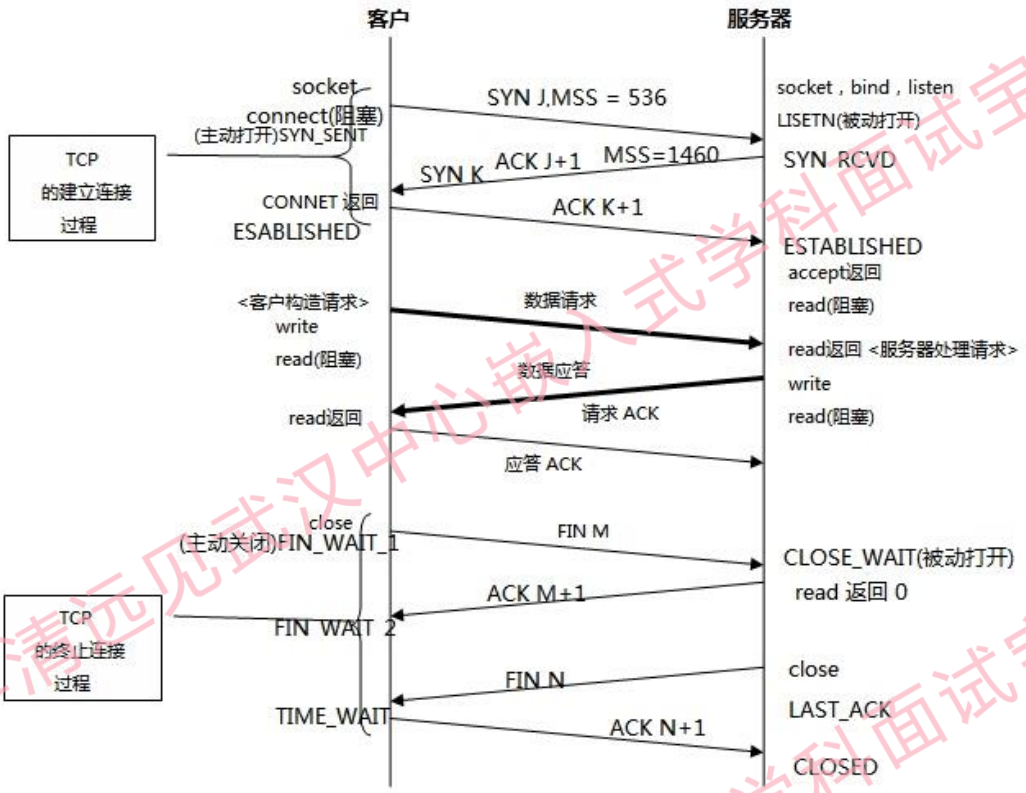
报文内容：包含 **ACK 标志位置为 1**，和客户端确认序列号（`ack = z + 1`）

目的：客户端确认连接关闭。之后，客户端和服务器都会关闭连接

5.8 TCP 中各个状态

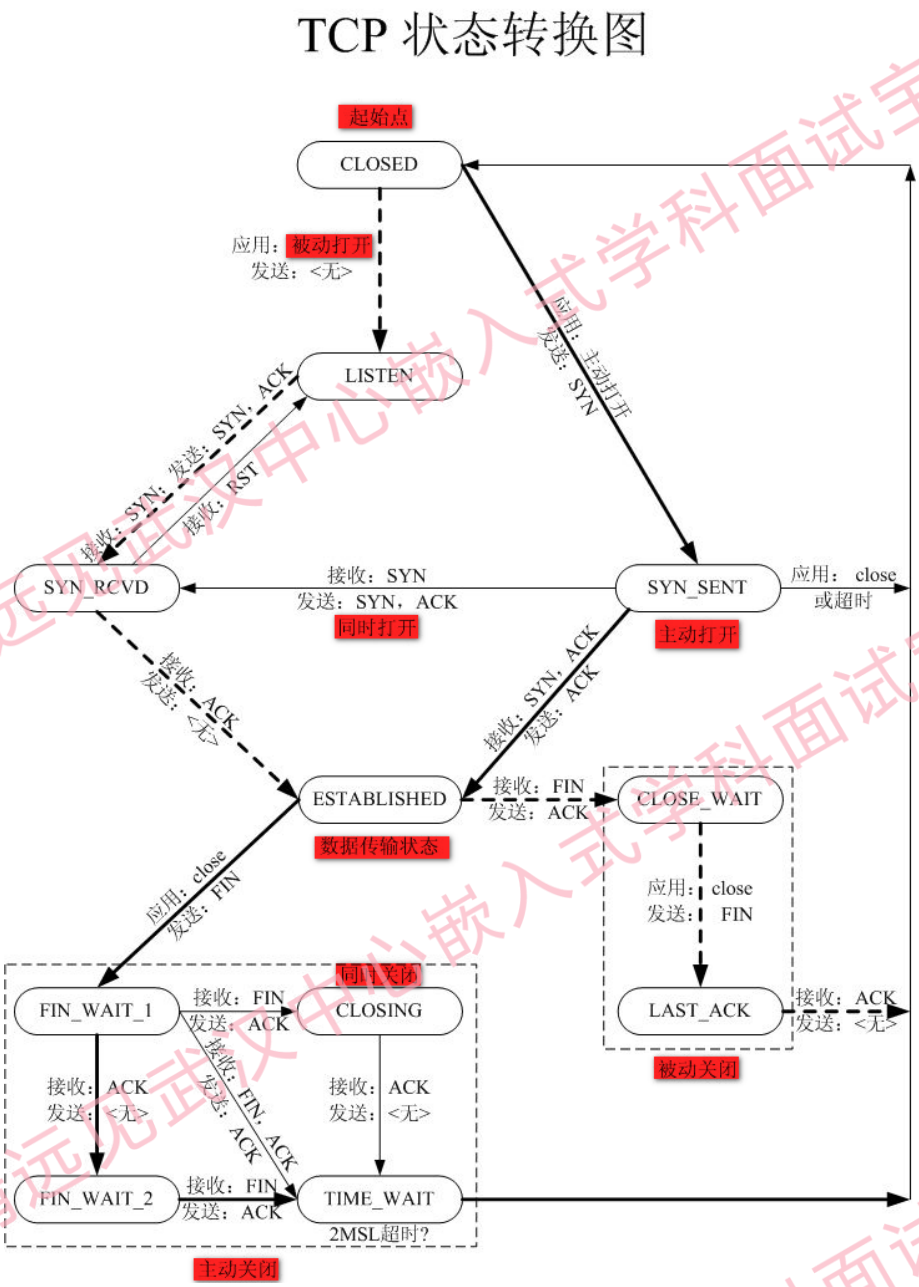
TCP 通信过程中各步骤的状态图 1

华清远见武汉中心嵌入式学科面试宝典



状态图 2

华清远见武汉中心嵌入式学科面试宝典



CLOSED： 表示初始状态。

LISTEN： 表示服务器端的某个 SOCKET 处于监听状态，可以接收连接了。

SYN_RCVD： 这个状态表示接收到了 SYN 报文，在正常情况下，这个状态是服务器端的 SOCKET 在建立 TCP 连接时的三次握手会话过程中的一个中间状态，很短暂，基本上用 netstat 你是很难看到这种状态的，除非你特意写了一个客户端测试程序，故意将三次 TCP 握手过程中最后一个 ACK 报文不予发送。因此这种状态时，当收到客户端的 ACK 报文后，它会进入到 ESTABLISHED 状态。

SYN_SENT： 这个状态与 SYN_RCVD 相呼应，当客户端 SOCKET 执行 CONNECT 连接时，它首先发送 SYN 报文，因此也随即它会进入到了 SYN_SENT 状态，并等待服务端的发送三次握手中的第 2 个报

华清远见武汉中心嵌入式学科面试宝典

文。SYN_SENT 状态表示客户端已发送 SYN 报文。

ESTABLISHED：表示连接已经建立了。

FIN_WAIT_1： FIN_WAIT_1 状态实际上是当 SOCKET 在 ESTABLISHED 状态时，它想主动关闭连接，向对方发送了 FIN 报文，此时该 SOCKET 即进入到 FIN_WAIT_1 状态。而当对方回应 ACK 报文后，则进入到 FIN_WAIT_2 状态，当然在实际的正常情况下，无论对方何种情况下，都应该马 上回应 ACK 报文，所以 FIN_WAIT_1 状态一般是比较难见到的，而 FIN_WAIT_2 状态还有时常常可以用 netstat 看到。

FIN_WAIT_2：实际上 FIN_WAIT_2 状态下的 SOCKET，表示半连接，也即有一方要求 close 连接，但另外还告诉对方，我暂时还有点数据需要传送给你，稍后再关闭连接。

TIME_WAIT： 表示收到了对方的 FIN 报文，并发送出了 ACK 报文，就等 2MSL 后即可回到 CLOSED 可用状态了。如果 FIN_WAIT_1 状态下，收到了对方同时带 FIN 标志和 ACK 标志的报文时，可以直接进入到 TIME_WAIT 状态，而无须经过 FIN_WAIT_2 状态。

CLOSING(图中没有标志这种状态)：这种状态比较特殊，实际情况中应该是很少见，属于一种比较罕见的例外状态。正常情况下，当你发送 FIN 报文后，按理来说是应该先收到（或同时收到）对方的 ACK 报文，再收到对方的 FIN 报文。但是 CLOSING 状态表示你发送 FIN 报文后，并没有收到对方的 ACK 报文，反而却也收到了对方的 FIN 报文。什么情况下会出现此种情况呢？其实细想一下，也不难得出结论：那就是如果双方几乎在同时 close 一个 SOCKET 的话，那么就出现了双方同时发送 FIN 报文的情况，也即会出现 CLOSING 状态，表示双方都正在关闭 SOCKET 连接。

CLOSE_WAIT：这种状态的含义其实是表示在等待关闭。怎么理解呢？当对方 close 一个 SOCKET 后发送 FIN 报文给自己，你系统毫无疑问地会回应一个 ACK 报文给对方，此时则进入到 CLOSE_WAIT 状态。接下来呢，实际上你真正需要考虑的事情是察看你是否还有数据发送给对方，如果没有的话，那么你也就可以 close 这个 SOCKET，发送 FIN 报文给对方，也即关闭连接。所以你在 CLOSE_WAIT 状态下，需要完成的事情是等待你去关闭连接。

LAST_ACK：它是被动关闭一方在发送 FIN 报文后，最后等待对方的 ACK 报文。当收到 ACK 报文后，也即可以进入到 CLOSED 可用状态了。

为什么主动方最后还需要经历 TIME_WAIT 状态才会关闭？

1、TIME_WAIT 状态需要经过 2MSL(两倍的最大报文段生存时间)才会回到 CLOSE 状态，由于网络本身不可靠，挥手最后一个 ACK 应答包可能会丢失，假设这个包丢失后，被动方会再次发送一个 FIN 挥手请

华清远见武汉中心嵌入式学科面试宝典

求包，此时如果主动方已经关闭连接，则被动方会认为连接出错。

2、相同的两个端可能在断开连接后紧接着再次握手连接，这时上次通信中的迷途数据包到达此端口则会被认为是这次的数据包。等待 2MSL 也可以让网络中所有的迷途数据包消亡，则不会影响后面的通信

5.9 TCP 协议和 UDP 协议的区别

	TCP	UDP
特点	**可靠通信，保证了数据的有序、完整、到达**	**不能保证数据有序、完整、到达**
面向连接	**面向连接**的，只有在连接建立后才能够进行通信；	**不是面向连接**的，任意UDP套接字只要获得了接收方的地址，就能够发送数据给接收方；
通信数据包	基于**字节流**的传输方式。数据以字节流的形式传输，并且会对数据进行分片和重组。每个数据包都有序列号，接收方按序接收。 对用户而言，单次通信数据包长度的不做限制，用户不用考虑拆包问题，而要考虑**粘包问题**。因为任意长度的数据都会被TCP低层算法拆包，以某个长度的小数据包发送给接收方。且连续2次通信是可能被接收方一次性接收处理。	基于**数据报**的传输方式。每个数据报都是一个独立的单元，彼此之间没有关系。数据报在传输时保留其独立性，接收方按数据报接收 单次通信数据会数据长度有限制，超过限制只能在用户层进行封包，并要设计协议告知接收方数据长度。 不存在粘包问题，但是存在丢包问题，因为UDP下每次通信数据包没有应答机制，除非在应用层手动实现应答（需要做同步）。
多连接与广播	需要用户自己逐步构建多连接，并由程序员使用一些手段进行维护，不支持通俗意义上的广播，最多能在现有的链接中进行“广播”。	本身没有限制通信规模，支持广播、多播、组播
服务器应用	服务通过套接字标识客户端的情况，对客户端的遍历或者发起通信，只需要通过相应的套接字（套接字集合）即可；	UDP一般不叫服务器，因为UDP是对等通信，不存在主机程序提供服务这一说。而且单次通信数据包中包含了发送方的地址信息，但在应用层中没有刻意存储客户端的地

华清远见武汉中心嵌入式学科面试宝典

		址信息。
应用场景	HTTP/HTTPS：用于网页浏览 FTP：用于文件传输 SMTP：用于电子邮件传输 SSH：用于安全的远程登录	DNS：域名解析 视频流：实时视频播放

5.10 如何保证 UDP 协议可靠

模拟 tcp 的可靠机制来保证四个无即可：无差错、无丢失、无重复、无失序；

1）数据包添加序号，保证单次通信的多个数据包之间有序；

2）应用层增加校验机制，如接收方计算数据包的校验码与数据包中提供的校验码不一致，则丢弃该数据包请求重发；

3）应用层增加确认机制，如果发送方没有在规定的时间内没有收到接收方的应答包，则进行重发，三次后还未收到应答直接判断发送失败；

5.11 TCP 断电现象及如何排查

TCP 断电现象指的是在 TCP 连接建立后，由于一方设备突然断电或网络中断，导致另一方在未收到断开连接通知的情况下，仍然认为连接是活跃的。这种情况会导致数据传输失败或网络资源占用未能及时释放；

断电现象表现：

连接"僵死"：客户端或服务器保持一个无效的连接，无法发送或接收数据。

未能及时释放资源：未检测到对方已经断开，导致资源（如 TCP 端口）被长时间占用。

数据传输失败：尝试发送数据但未收到确认，导致数据丢失或重传失败。

排查方法：

超时机制与心跳检测：TCP 协议提供了 Keep-Alive 功能，通过定期发送探测报文来确认对方是否仍然在线。如果在多次探测后未收到响应，TCP 将认为连接已断开并释放资源；在应用层设计心跳机制，定期发送心跳包确认对方是否在线。若未收到心跳响应，则主动关闭连接或重试连接；

重传机制与超时机制：CP 具有内置的重传机制，如果在规定时间内未收到数据包的 ACK 确认，TCP 将自动重传数据包。配置合理的重传次数和超时设置可以帮助检测断电现象；配置应用程序在一定时间

华清远见武汉中心嵌入式学科面试宝典

未能收到响应时，主动关闭连接并尝试重新建立连接

分析 TCP 状态：使用 netstat 工具查看 TCP 连接状态，识别是否有大量处于`CLOSE_WAIT`、`FIN_WAIT`、`TIME_WAIT`等状态的连接，这些状态可能表示存在断电引起的连接问题；抓包工具，分析 TCP 相关数据包状态

5.12 什么是长连接、短连接

长连接：

定义：长连接是一种在初次建立连接后，保持连接不断开，允许多次数据传输的连接方式。连接在完成一次数据传输后并不立即关闭，而是继续保持打开状态，以便后续的请求可以复用该连接

工作原理：

客户端与服务器建立连接后，不会立即关闭连接，而是保持连接打开。

后续的请求可以复用该已经建立的连接，从而避免了频繁的连接建立和断开。

长时间未传输数据时，服务器或客户端可能会关闭连接以节省资源，或通过发送心跳包来保持连接活跃。

优点：

减少连接开销：不需要频繁建立和关闭连接，节省了 TCP 三次握手和四次挥手的开销

提高传输效率：适用于需要频繁通信的场景，比如聊天应用连接等。

缺点：

资源占用：长时间保持连接会占用服务器资源，尤其是在并发量高的情况下。

可能导致服务器压力过大：需要对连接进行管理，以避免资源耗尽。

短连接：

定义：短连接是一种在完成一次数据传输后，立即关闭连接的方式。每次请求都需要重新建立连接，数据传输完毕后随即关闭

工作原理：

客户端发起请求时，与服务器建立连接。

数据传输完成后，立即关闭连接。

每次新的请求都会重新建立连接。

优点：

华清远见武汉中心嵌入式学科面试宝典

节省资源：每次通信结束后立即释放连接资源，适用于连接数多但每次请求时间短的场景。

管理简单：因为连接很快关闭，服务器不需要长期管理大量的连接状态。

缺点：

开销较大：每次请求都需要重新建立和断开连接，增加了连接的开销。

传输效率较低：对于频繁请求的场景，频繁建立和关闭连接会导致效率降低。

5.13 简述 select、poll、epoll 原理、方式，以及三种之间的区别

	select	poll	Epoll（**Linux 特有**）
工作原理	Select 通过一个**固定大小的位掩码（位图）**来监控文件描述符集合。应用程序需要将感兴趣的文件描述符（如 socket）添加到一个 fd_set 中，然后调用 select()来检测哪些文件描述符有事件发生	Poll 类似于 select，但它使用一个**结构体数组**来存储文件描述符及其事件。应用程序通过设置`pollfd`结构体数组来监控文件描述符	Epoll 专为**大规模文件描述符监控设计，效率更高**。它使用一个`epoll`实例来管理文件描述符集合。应用程序首先使用`epoll_ctl()`注册文件描述符到`epoll`实例，然后调用`epoll_wait()`来等待事件的发生。 Epoll 通过**事件驱动机制**，只在有事件发生时通知应用程序，而不是遍历所有文件描述符。
事件集合	用户通过 3 个参数分别传入感兴趣的**可读、可写、异常**事件集合。内核在**遍历修改集合以便反馈其中的就绪事件**。这使得用户每次调用 select 都要**还原**这 3 个事件集合。	**统一处理**所有的事件类型，只有一个事件集合参数。用户通过**pollfd.enevts 传入**关心去的时间，内核修改**pollfd.revemts 反馈**其中的事件。	内核通过一个事件表直接管理多文件的 IO 事件。因此每次调用 epollwait 时，无须反复传入用户感兴趣的事件。epoll_wait 的参数 events 仅用来反馈就绪事件。

华清远见武汉中心嵌入式学科面试宝典

时间复杂度	**O(n)**	**O(n)**	**O(1)**
触发	水平触发模式 LT	水平触发模式 LT	水平/边沿触发模式：LT/ET
优缺点	每次调用 select，都需要把 fd 集合从用户态拷贝到内核态，且都需要在内核遍历传递进来的所有 fd，开销在 fd 很多时也很大；select 支持的文件描述符数量最多是 **1024**；且只支持水平触发；	poll 的实现和 select 非常相似，只是描述 fd 集合的方式不同，poll 使用结构体 pollfd 而不是 select 的事件集合 fd_set，因而没有文件描述符数量限制，其他的都差不多。	epoll 保证了每个 fd 在整个 IO 轮询过程中**只会拷贝一次，不存在重复操作；** epoll 没有文件描述符数量限制，它所支持的 FD 上限是最大可以打开文件的数目，这个数字一般远大于 2048；

三者之间的区别

1．性能与效率：

Select：由于需要遍历整个文件描述符集合，性能较低，适合少量文件描述符的场景。文件描述符数量受限。

Poll：性能稍优于 select，但仍需遍历整个文件描述符数组。没有文件描述符数量的限制，但每次调用都要传递整个数组。

Epoll： 性能最优，尤其适用于大量文件描述符的场景。采用事件通知机制，不需遍历整个集合，适合高并发场景。

2．扩展性

Select：扩展性差，受限于`FD_SETSIZE`，不能高效处理大量连接。

Poll：比 select 更灵活，但扩展性仍有限。

Epoll：扩展性最好，可以轻松处理数以万计的文件描述符。

3．使用复杂性：

Select：最简单，但在处理大量文件描述符时效率低。

Poll：使用复杂度适中，稍微比 select 更复杂。

Epoll：最复杂，尤其是在 ET 模式下，需要更多的代码管理和精细控制。

华清远见武汉中心嵌入式学科面试宝典

总结

Select： 适用于小规模、简单的场景。由于受限于文件描述符数量和需要每次遍历整个集合，效率较低。

Poll：在文件描述符数量大于 1024 时，表现优于 select，但在处理大量连接时仍然效率较低。

Epoll：适用于高并发、大量连接的场景，尤其是在 Linux 下的服务器应用程序中最为常用。它的高效事件通知机制使得它在处理大量 I/O 事件时具有显著优势。

5.14 简述三种超时检测的方法（设置套接字属性，使用 select 函数，使用定时器信号）

设置套接字属性

方法：通过直接设置套接字的超时属性来实现超时检测。这种方法直接影响套接字的行为，例如设置发送或接收操作的超时时间

实现步骤：

使用`setsockopt()`函数为套接字设置超时参数

可以设置发送超时（`SO_SNDTIMEO`）和接收超时（`SO_RCVTIMEO`）两种属性

优点：

实现简单，直接影响套接字的操作，适用于需要对特定套接字操作进行超时控制的场景

缺点：

仅对单个套接字操作生效，不能精细控制整个应用的超时行为

使用 select 函数

方法：`select`函数可以监控多个文件描述符（包括套接字）的可读性、可写性和异常状态，并设置超时时间。在超时时间内，如果没有任何文件描述符发生变化，`select`函数将返回 0

实现步骤：

将套接字添加到`fd_set`集合中，然后调用`select()`函数。

通过设置`timeval`结构体来指定超时时间。

优点：

能同时监控多个文件描述符的状态，并且支持读、写、异常的多种监控模式

可用于实现更复杂的 I/O 多路复用场景

缺点：

华清远见武汉中心嵌入式学科面试宝典

实现复杂度较高，且在处理大量文件描述符时效率可能不如`epoll`等更高级的机制

使用定时器信号

方法：使用定时器信号（如`SIGALRM`）来实现超时控制。通过在一定时间内发出定时器信号，可以中断当前的阻塞操作（如`recv()`或`send()`），从而实现超时检测

实现步骤：

设置定时器信号处理函数

使用`alarm()`或`setitimer()`函数来设置超时时间

优点：

可以中断当前的系统调用，从而实现对阻塞操作的超时控制

适用于无法直接设置超时属性的操作，或需要全局统一管理超时的场景

缺点：

使用信号会引入异步处理逻辑，可能导致代码复杂度增加

信号处理的可靠性和实时性依赖于系统的实现

5.15 Sqlite3 数据的常用指令

数据库操作

创建数据库	sqlite3 database_name.db
打开数据库	.open database_name.db
关闭数据库	.exit

表操作

创建表	CREATE TABLE table_name (column1 datatype PRIMARY KEY, column2 datatype, column3 datatype);
删除表	DROP TABLE table_name;

华清远见武汉中心嵌入式学科面试宝典

数据操作

插入数据	INSERT INTO table_name (column1, column2) VALUES (value1, value2);
删除数据	DELETE FROM table_name WHERE condition;
更新数据	UPDATE table_name SET column1 = value1 WHERE condition;
查找数据	SELECT column1, column2 FROM table_name WHERE condition;

5.16 Sqlite3 数据库 API 函数

数据库连接与关闭：

函数原型：int sqlite3_open(const char *filename, sqlite3 **ppDb); 功能：打开一个 SQLite 数据库文件。如果数据库文件不存在，`sqlite3_open()` 会创建一个新的数据库文件 参数： filename：数据库文件名。 ppDb：用于存储数据库连接句柄的指针。 返回值：成功返回 `SQLITE_OK`，失败返回错误码
函数原型：int sqlite3_close(sqlite3 *db); 功能：关闭数据库连接，释放资源 参数： db：数据库连接句柄 返回值：成功返回 `SQLITE_OK`，失败返回错误码

SQL 语句执行

函数原型：int sqlite3_exec(sqlite3 *db, const char *sql, int (*callback)(void*,int,char**,char**), void *data, char **errmsg); 功能：执行一条 SQL 语句，包括`CREATE`, `INSERT`, `UPDATE`, `DELETE`, `SELECT`等

华清远见武汉中心嵌入式学科面试宝典

参数： db：数据库连接句柄。

sql：要执行的 SQL 语句。

callback： 当 SQL 语句执行完毕后调用的回调函数，用于处理查询结果。可以为`NULL`。

data：传递给回调函数的用户数据。

errmsg：如果执行失败，该指针将指向错误消息字符串。

返回值： 成功返回 `SQLITE_OK`，失败返回错误码

```
函数原型：int sqlite3_get_table(
            sqlite3 *db,              /* An open database */
            const char *zSql,      /* SQL to be evaluated */
            char ***pazResult,    /* Results of the query */
            int *pnRow,               /* Number of result rows written here
        */
            int *pnColumn,         /* Number of result columns written here
*/
            char **pzErrmsg    /* Error msg written here */
    );
```

功能：查询表数据

参数： db:打开的数据库

zSql:sql 语句

pazResult:获得查询结果存放的指针数组首地址

pnRow:获得查询记录的条数

pnColumn:查询到列的个数

pzErrmsg:获得错误信息

返回值:成功返回 `SQLITE_OK`，失败返回错误码

错误处理

```
函数原型：const char *sqlite3_errmsg(sqlite3 *db);
```

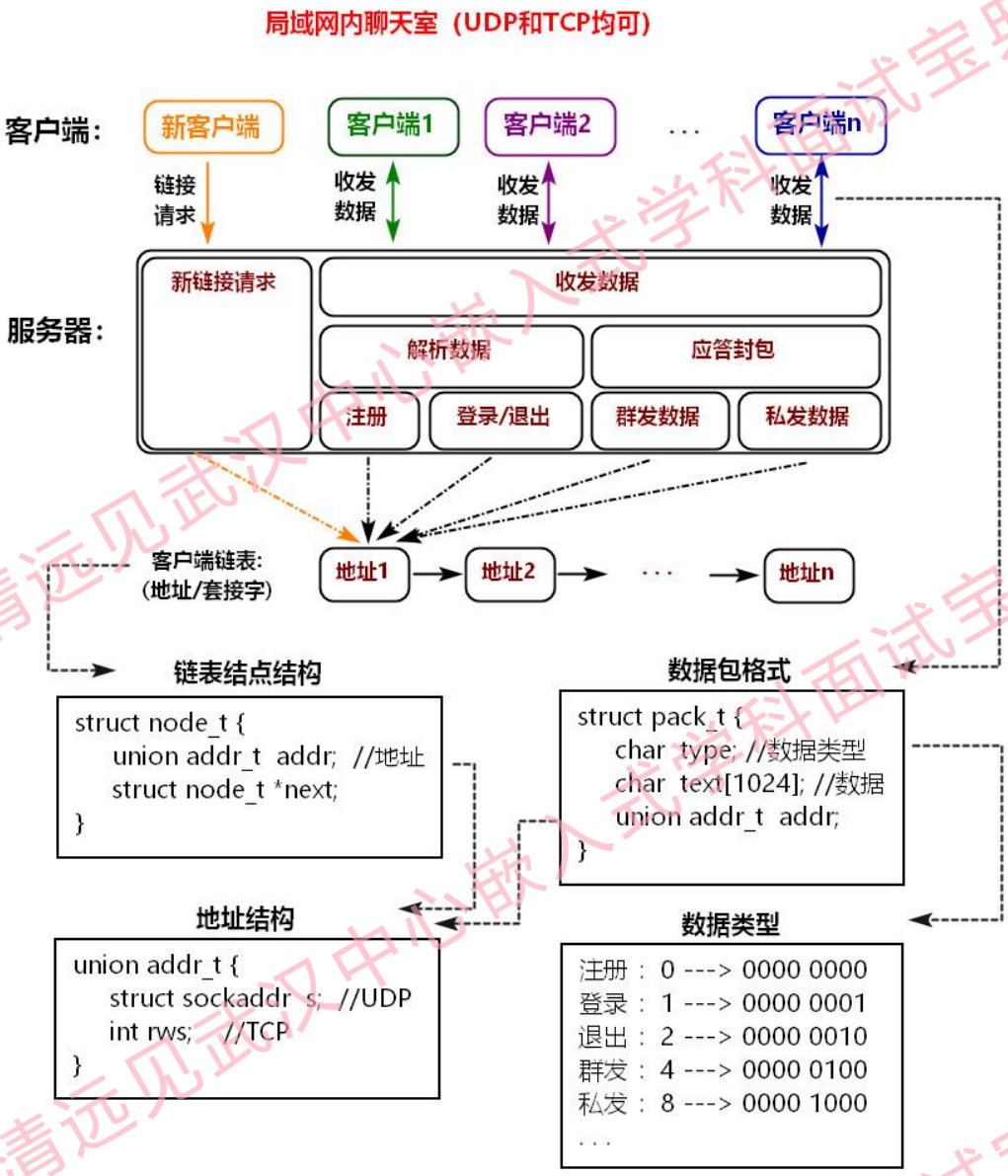
功能：获取最近一次 SQLite 操作的错误消息字符串

华清远见武汉中心嵌入式学科面试宝典

参数： db：数据库连接句柄

5.17 局域网内聊天室的项目框架是什么样的

整个项目分为客户端与服务器，通过局域网群发及私发通讯。项目涉及的框架及数据结构如下图所示：



客户端：主要功能是向局域网内服务器发送协议数据，以实现登录、注册、群发、私发等功能；其特点在于和用户的交互，并采用 2 个线程协同：主线程负责与用户交互，等待用户选择功能；而子线程负责监听套接字数据，并在适当的时机打印，以实现群聊效果；

服务器：为多个客户端服务，建立起类似于组播/单播的机制，具体实现细节如下：

1）网络通讯模块是基于 Linux socket 接口实现，使用 udp 协议/TCP 协议均可。但 udp 协议非

华清远见武汉中心嵌入式学科面试宝典

面向连接的协议，传输速度快，缺点是客户端是否在线是否收到数据，服务端并不知道。我们这里没有设计心跳包及应答机制，暂且忽略。（使用 TCP 则没有这个问题）

2)服务器上有一个链表，其功能是保存客户端的地址信息，包括客户端的 ip 地址与端口号。为服务器遍历活跃客户端提供了可能，同时客户端的注册、登录、退出、群发都需要依赖服务器上的链表数据；

3）为提高服务器性能，采用了多线程并发方案，配合读写锁互斥，使得多个线程都能单独完成服务器中的注册、登录、退出、群发、私发。同时接收多客户端数据时利用套接字本身的缓冲属性存储客户端请求数据，然后由主线程获取单个单个的数据包，启动相应的线程完成相应的功能。

5.11 网络电子词典是如何实现的



(1) 网络电子词典项目是一款基于网络的以单词查询为主要功能的软件,具体实现功能如下:

单词查询,账号注册与登录,历史记录管理,客户端掉线检测

(2) 整个项目是基于 C/S 架构，分为客户端与服务器,具有如下特点:

(a) 支持高并发,使用多进程的方案

(b) 高可靠性，在应用层基于自定义协议,在底层基于 tcp 协议

(3) 项目主要分为如下模块:

(a) 网络通讯模块,传输层基于 tcp 协议,应用层基于自定义协议

(b) 数据持久化模块,使用轻量级数据库 SQLITE3

(c) 客户端菜单模块,使用命令行界面

(4) 网络模块采用 tcp 协议与自定义协议，以保证数据通讯可靠性,支持客户端掉线检测

华清远见武汉中心嵌入式学科面试宝典

(5) 数据持久化模块使用了 SQLITE3 数据库,数据内容由 user 表与 dict 表来管理

a) user 表，用于管理用户账号信息；

b) dict 表，用于管理单词信息

(6) 客户端菜单模块，基于命令行界面，使用了二级模式,在登录成功后，可以进行相应的单词查询